



Department of Electrical and Information Engineering
Faculty of Engineering, University of Ruhuna

Mini Project Report
EC8203 Applied Big Data Engineering

Ride-Hailing Fleet Operations

A Lambda architecture for live fleet status and daily profitability

EG/20XX/XXXX JATHURSHAN

EG/20XX/XXXX RIFAT

EG/20XX/XXXX SHAMIL

Computer Engineering

September 28, 2026

Source code: <https://github.com/jathurT/ride-hailing-lambda>

Contents

1	The problem	2
2	Requirements	2
3	Lambda or Kappa	2
4	Architecture	4
5	Technology choices	6
6	Implementation	8
7	Observability	10
8	Results	11
9	Limitations and lessons	14
10	Conclusion	16
	References	16
A	Individual contributions	16
B	Reproducing the results	16
C	More screenshots	16

1. The problem

A ride-hailing company runs a fleet of vehicles. Every vehicle sends its position, speed and status all the time. Once a day, the fuel and garage partners send a file with what each vehicle cost to run. Dispatch staff want to see the fleet as it is now. Finance wants to know which vehicles are losing money once those costs are counted. This project builds one data platform that answers both questions from the same data. The assignment states the question like this:

Business question (use case 1)

What is fleet utilization and earnings by area and time of day right now, and which vehicles are becoming unprofitable once yesterday's fuel and maintenance costs are factored in?

This one sentence holds two questions, and they want opposite things.

Question A, “right now”, is asked by dispatch, all day, on a wall screen. The answer must arrive within seconds. A small error does not matter: nobody decides anything differently because utilization is 71 % and not 72 %.

Question B, “becoming unprofitable”, is asked by finance, once a day. The answer can take hours, and it has to: the partner cost file does not exist until the day is over. But the answer must be exact, because it is used to settle with drivers. The word “becoming” also makes it a question about a trend over several days, not about one day.

One question accepts error and needs speed. The other accepts delay and needs correctness. A single processing path would have to give up one of them. That is the reason, argued in Section 3, that the system is a Lambda architecture.

1.1. The two data sources

Both sources are Python programs that simulate the real thing.

- **Streaming source: vehicle telemetry.** 150 vehicles move around a grid of 12 zones in the city. Each vehicle is *idle*, *on the way* to a pickup, or *on a trip*, and demand rises and falls through the day. Every vehicle sends one reading every 3 simulated minutes, about 239.9 events per real second.
- **Daily batch source: partner expenses.** One CSV file per simulated day with fuel and maintenance cost per vehicle, written straight to object storage and never through Kafka. The dropper also produces the awkward cases: a missing file and a corrected file for an earlier day.

Figure 1 shows who sends data into the platform and who uses what comes out. It names no technology on purpose.

2. Requirements

We turned the question into requirements before choosing any technology. The non-functional targets are numbers, so that Section 8 can say whether each one was met.

Simulated time

One simulated day lasts **300 real seconds** (SIM_DAY_SECONDS=300), so the clock runs **288 times** faster than real time. The simulation starts on 1 March 2026. Every container reads simulated time from one shared file written once at start-up, so no two services can disagree about the date. In this report, a time is marked *real* or *simulated* whenever it could be read either way. For example, the 30 simulated minute watermark is 6.25 real seconds.

3. Lambda or Kappa

We compared the two architectures on the eight criteria from the module. Six favour Lambda, including the two we think decide the question. Two favour Kappa, and we accept both.

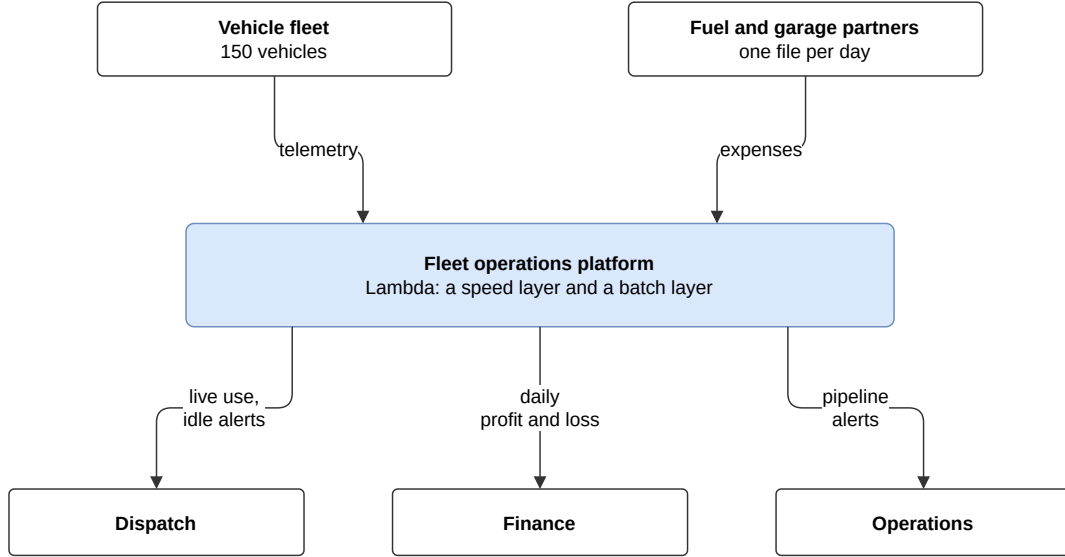


Figure 1: System context. One source is continuous and one is a daily file. Dispatch gives up some accuracy for speed; finance gives up speed for accuracy.

ID	Requirement	Where it is met	Target
FR-1	Live fleet utilization by zone	Speed layer, <code>/api/v1/fleet/live</code>	
FR-2	Earnings by area and time of day	Exact hourly table and live view	
FR-3	Daily profit per vehicle after costs	Batch layer, daily P&L table	
FR-4	Vehicles becoming unprofitable	7-day rolling average and trend slope	7 sim days
FR-5	Alert on vehicles idle for too long	Stateful streaming detector	45 sim min
FR-6	One consolidated daily view	Grafana dashboards	once a day
NFR-1	Freshness of the live view	10 s micro-batch trigger	under 30 s
NFR-2	Accuracy of profitability	Exact recompute from stored raw data	exact
NFR-3	Corrected files handled safely	Upsert per vehicle and date	idempotent
NFR-4	Pipeline failure detected	Prometheus alert rules	under 3 min
NFR-5	Easy to reproduce	Docker Compose	one command

Table 1: Requirements and where each one is met.

Criterion	Favours	Reason for this use case
Latency	Lambda	Two latency needs at the same time: seconds for dispatch, a day for finance. The daily delay comes from the partners, not from us.
Data volume	Lambda	72,000 events per simulated day here; a real fleet of 5,000 vehicles sends 14.4 million a day, and a trend needs 7 to 30 days of it at rest.
Historical analysis	Lambda	Decisive. “Becoming unprofitable” is a trend over many days. Holding weeks of state in a stream job is fragile and is lost on restart.
Reprocessing	Lambda	Decisive. A corrected cost file for one day re-reads one stored day. Under Kappa it means replaying that whole day of telemetry.
Fault tolerance	Lambda	If the stream job fails, only the live screen goes stale. The financial record is rebuilt from stored raw data.
Accuracy	Lambda	The question itself asks for fast and approximate plus slow and exact.
Complexity	Kappa	Accepted. Reduced by one shared transform library and one merge point.
Cost	Kappa	Accepted for the number of containers; not for storage (see below).

Table 2: The eight criteria from the module, applied to this use case.

3.1. The two criteria that decide it

Historical analysis. Finance does not ask which vehicles lost money yesterday. It asks which vehicles are getting worse. We compute a 7 simulated day rolling net profit and a least-squares trend line for every vehicle, and label each one **HEALTHY**, **WATCH** or **UNPROFITABLE**. To do this from a stream, we would need to keep weeks of state inside the streaming job, or keep weeks of data in Kafka and replay it. Both bring back, in a worse form, the batch processing Kappa is meant to remove.

Reprocessing. Partners really do send corrections: a fuel card charged to the wrong vehicle, a garage bill a day late. Under Lambda, the corrected file lands, the batch job for that one date runs again, reads one stored day of telemetry, joins it again and overwrites the rows for that date. Section 8 shows that running it twice changes nothing. Under Kappa the same correction means replaying every telemetry event of that day. The cost of a correction should depend on the size of the correction, not on the size of the biggest dataset.

3.2. The two criteria we accept

Complexity. The module names two Lambda drawbacks: two codebases, and keeping two systems in agreement. There is only one codebase for business logic: every rule lives in `src/fleet/transforms/` as plain functions on DataFrames, which the speed layer calls on each micro-batch and the batch layer on a whole day. They are tested once, and an architecture test fails if either layer defines its own logic. And the two views are not hoped to agree: Section 4.1 defines exactly where one stops.

Cost. Lambda runs more containers. But the history is Parquet on object storage, the cheapest durable storage, while Redis holds only two simulated hours, and Kafka keeps telemetry for one real hour. A Kappa system that replays 30 days must keep 30 days on the brokers, which costs more per byte. So we accept the cost criterion for containers, but not for storage.

3.3. The case for Kappa, and why it lost

Kappa has a real case. One path, one codebase, and nothing for two views to disagree about. Fewer containers, no object store, no watermark bookkeeping. The expense file could even be a stream: publish each row to a compacted topic keyed by vehicle and date, so that a correction is simply a newer record under the same key, and join it with telemetry continuously. At our volume, replay would be cheap.

It lost for four reasons. History is the question, not a side feature, and the module itself says Kappa is weaker at historical analysis. Reprocessing cost points the wrong way, and corrections happen every week. Profit per vehicle is used to pay drivers, and a stored file plus the job that produced a figure is a better answer to an auditor than a replayed log. And a CSV that a garage uploads once a day is not an event stream.

What would change our minds. We would choose Kappa if all three of these became true: costs arrived continuously (for example from fuel sensors) instead of in a daily invoice; the trend window became short enough to keep in stream state (hours, not weeks); and nobody needed to audit the financial figures.

4. Architecture

Four properties of Figure 2 carry the design.

- **One log, several independent readers.** Kafka keeps telemetry as a stored log, not a queue. Six streaming queries read the same records, each with its own position and checkpoint, so one can fail without stopping the others. A Redis outage does not stop the Parquet writes.
- **The raw data is never changed.** Valid telemetry is appended to the lake as Parquet, one folder per simulated date. The batch layer only reads from it, so a correction is always a recomputation, and any past day can be reproduced.
- **The zone is worked out when reading, not stored.** Telemetry is stored with its latitude

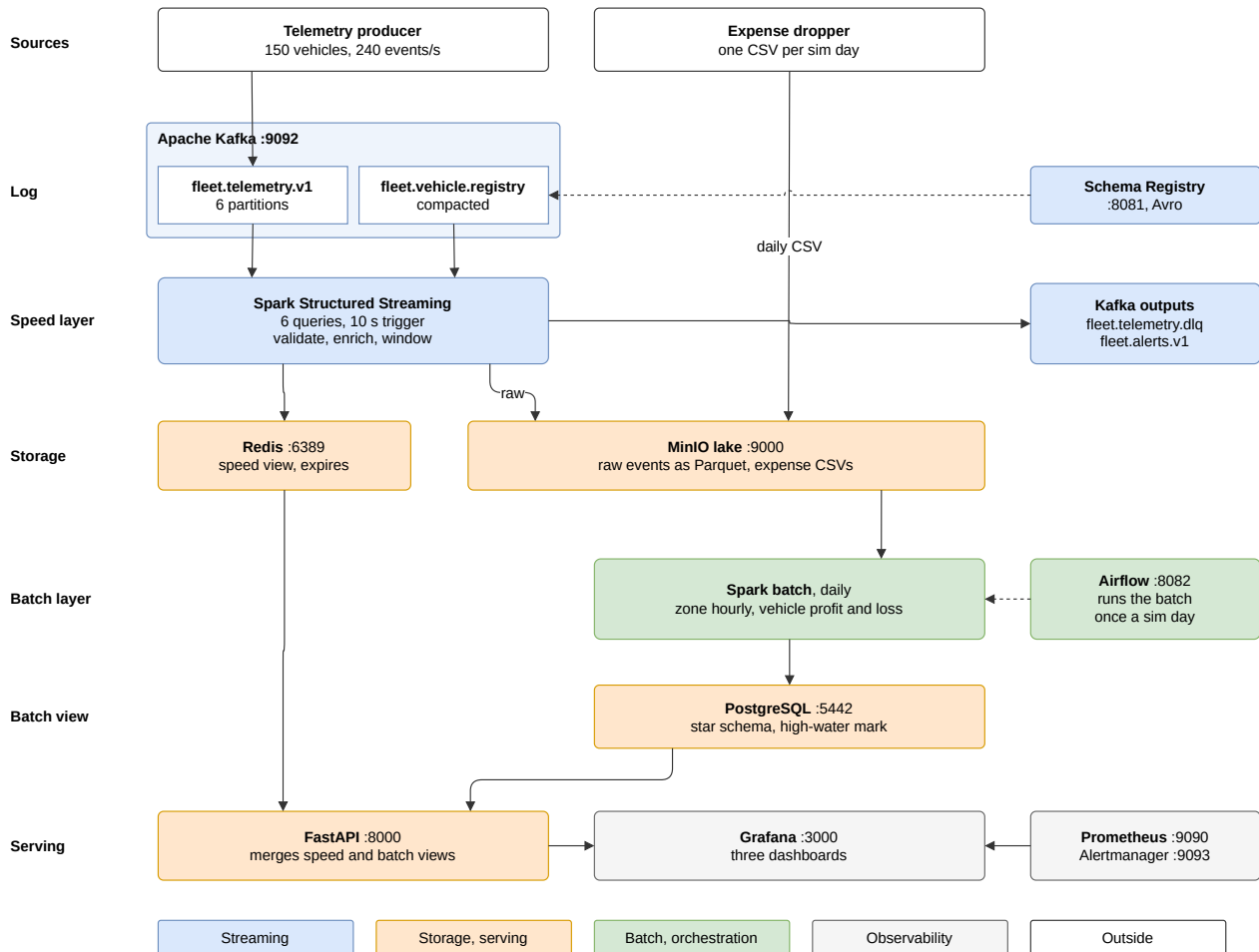


Figure 2: The layered architecture, read from top to bottom. Blue is the speed path, green the batch path and orange the serving layer where the two meet. The daily expense file goes straight to object storage, not through Kafka.

and longitude, and both layers use the same function to find the zone. If the zone grid is redrawn, history can be recomputed with the new grid.

- **The two paths meet in one place only:** the merge in the serving layer, described next.

4.1. The merge boundary

The batch layer keeps a one-row table, `batch_high_water_mark`, with the last simulated date it has fully processed. The serving layer reads it for every request and applies one rule:

The merge rule

The batch view answers every date up to and **including** the high-water mark. The speed view answers every date **after** it. So each date belongs to exactly one side: no date is counted twice and none is lost. Every row says which side produced it (**source**) and whether it is exact (**exact**).

The high-water mark moves inside the same database transaction as the facts it describes, so it can never point at a day whose data has not been saved.

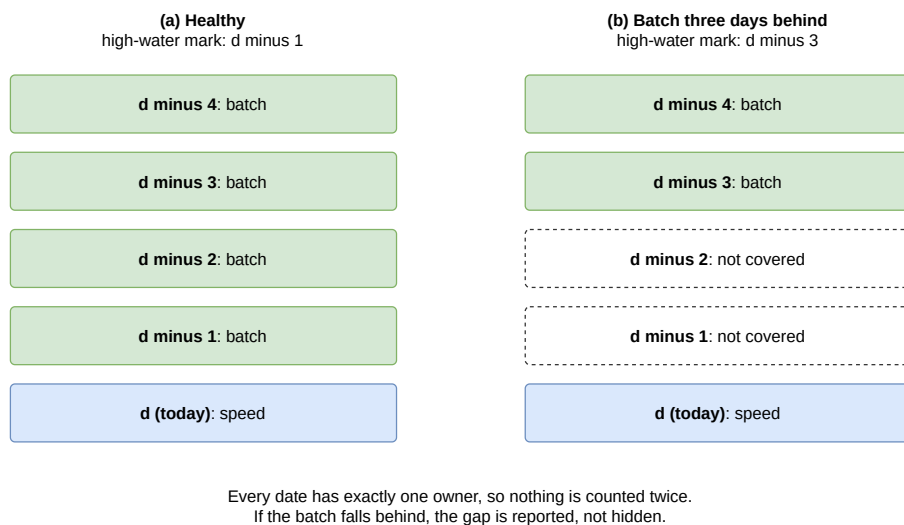


Figure 3: The merge rule. (a) Normal case: batch up to yesterday, speed for today. (b) Batch three days behind: the days in between are listed as uncovered, not silently dropped.

Two behaviours were added after testing. First, Redis only holds the current window, not a history per date. If the batch layer falls more than one day behind, some dates are answered by neither side. Returning fewer rows without a word would make a stalled pipeline look like a quiet week, so the response lists those dates in `uncovered_dates` and says why (Figure 3b). Second, a date in the future is reported as “not simulated yet”, not as “batch is behind”, so nobody goes looking for a fault that does not exist.

4.2. One event through the system

5. Technology choices

Each choice is justified for this use case, together with what we gave up for it.

Spark was chosen over Storm, Flink or Kafka Streams not for speed (any of them handles 240 events per second) but because it runs the *same code* in streaming and batch mode, which is how we keep one shared transform library. **Redis** was chosen partly because it is not durable: the live view must be disposable, and expiry is a property of the store. **Parquet on object storage** holds the raw data because it must never change; append-only files make “recompute that day” the only way to correct anything.

MinIO, Parquet, Prometheus, Grafana and Alertmanager were not covered in the module. We added them on purpose and say so here.

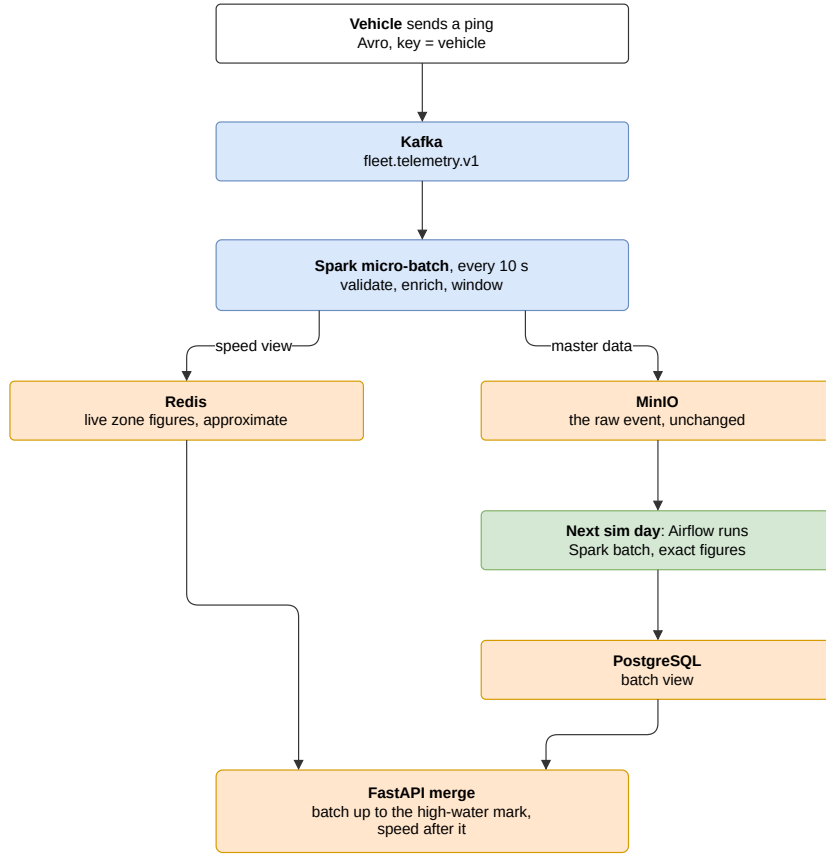


Figure 4: One telemetry event, end to end. The same event is counted approximately within seconds by the speed layer and stored unchanged for the exact batch run a simulated day later.

Layer	Chosen	Why, for this use case	What we gave up
Ingestion	Kafka (KRaft)	Order is kept per vehicle by keying on <code>vehicle_id</code> ; one log serves many readers	One broker, so no fault tolerance
Serialization	Avro and Schema Registry	The schema will change; much smaller than JSON	Harder to read by eye; one more container
Speed processing	Spark Structured Streaming	Same engine as the batch layer, so one shared transform library	Micro-batch delay
Speed store	Redis	Key lookups only; built-in expiry keeps the view small	Not durable, on purpose
Raw data store	MinIO and Parquet	Files are never changed, so a recompute gives the same result; column pruning	No transactions on the lake
Batch processing	Spark (batch)	Shares code with the speed layer; handles the wide join	JVM start-up for each run
Batch store	PostgreSQL star schema	Transactions for money; window functions for the rolling trend	Holds only aggregates
Orchestration	Airflow	File checks, retries, branching, and a UI that shows health	The heaviest component
Serving	FastAPI	A place for the merge rule to live and be tested	Our own code to maintain
Dashboards	Grafana	Set up from files, so a fresh clone gets the same dashboards	Less custom than a hand-made UI
Metrics and alerts	Prometheus, Pushgateway, Alertmanager	The required alert rules are one line of PromQL each; real routing and inhibition	Pushgateway keeps old values (Section 7)
Logging	<code>structlog</code> to JSON	Every log line has the same fields	

Table 3: Technology stack.

6. Implementation

6.1. Ingestion

The **partition key is the most important choice in this layer**. The idle detector keeps state per vehicle and must see *on trip*, then *idle*, then *idle* in that order. Kafka only keeps order inside one partition, so the key is `vehicle_id`: all readings of one vehicle go to one partition and arrive in order. Any other key would break the feature.

Messages are Avro, registered in the Schema Registry, in the Confluent wire format (a magic byte and a four-byte schema id in front of each message), so producers and consumers can change separately. A decoded message is shown in Figure 12 in the appendix.

Topic	Partitions and policy	Purpose
fleet.telemetry.v1	6, delete after 1 real hour	Vehicle readings, keyed by vehicle
fleet.telemetry.dlq	1, delete after 14 days	Rejected readings, original fields and the reason
fleet.alerts.v1	3, delete after 7 days	Idle-vehicle alerts from the speed layer
fleet.vehicle.registry	3, compacted	One record per vehicle, newest kept

Table 4: Kafka topics.

About 1 % of readings are broken on purpose: negative fares, missing coordinates, impossible speeds, positions outside the city and timestamps in the future. They are not decoration. They are the known input that lets us check the dead-letter path (Section 8).

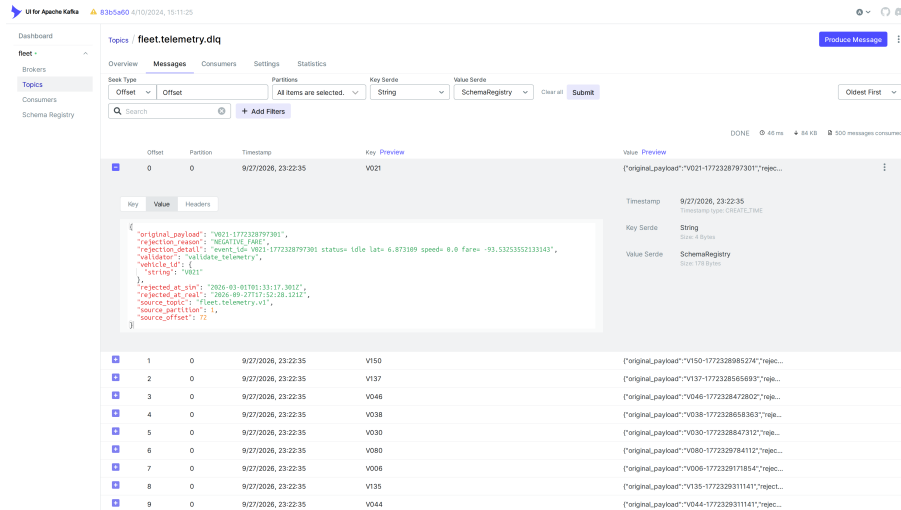


Figure 5: Kafka UI: the dead-letter topic. Each record keeps the original values and the reason it was rejected.

6.2. Speed layer

One Spark Structured Streaming job runs six queries on the telemetry topic, each with its own checkpoint, every 10 real seconds: the raw data writer, the dead-letter writer, zone activity, zone earnings, the idle detector, and live state per vehicle.

Two details matter. *Activity and earnings are separate queries*, because a streaming query allows only one aggregation. And *earnings must be computed from one row per trip*: the fare is repeated on every reading of a trip, so summing it directly multiplies revenue by the number of readings. This was found only by comparing total revenue with the number of distinct trips; nothing raised an error. The running queries and one zone of the live view in Redis are in Figures 13 and 14 in the appendix.

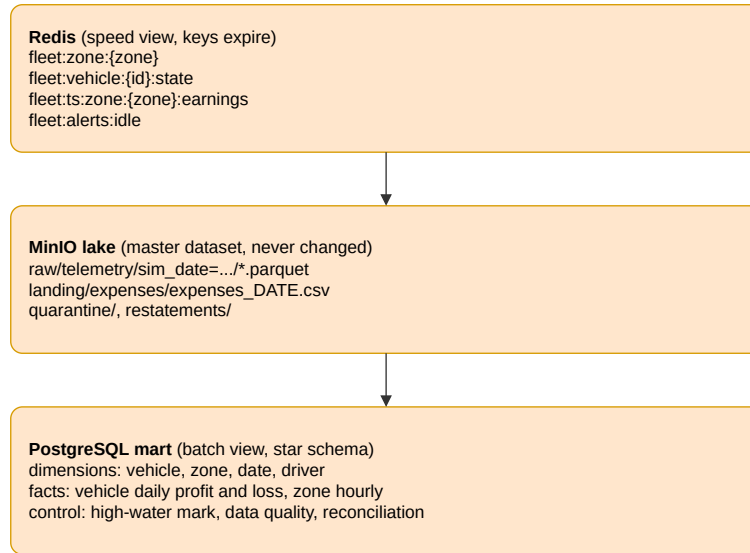


Figure 6: The three stores: Redis keys for the live view, the Parquet lake split by simulated date, and the PostgreSQL star schema for the batch view.

6.3. Storage

The lake keeps only valid readings, one folder per simulated date, so the batch job for one day reads exactly one folder. PostgreSQL holds the exact results in a star schema: a vehicle dimension, a daily P&L fact table keyed by vehicle and date, an hourly zone fact table, and the one-row high-water mark. Figure 15 in the appendix shows the date folders in the lake.

6.4. Batch layer and orchestration

Two Spark jobs run for each simulated day. The zone job computes exact hourly figures per zone with an exact distinct count, where the speed layer uses an approximate one; both sit in the same source file, so the trade-off is easy to see. The profitability job rebuilds trips, measures distance from our own GPS trace (haversine between consecutive positions), joins the expense file, and computes the 7-day rolling average and trend. The join keeps every case: vehicles with costs, vehicles with no cost row, and cost rows with no telemetry.

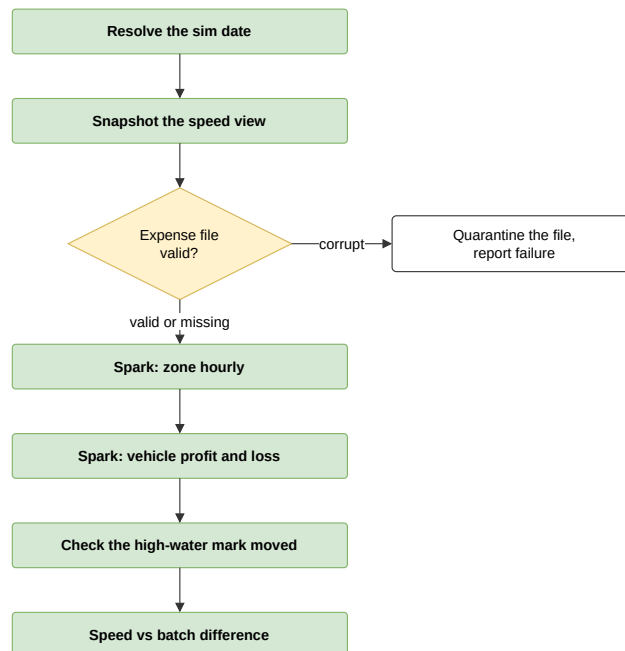


Figure 7: The daily DAG. A corrupt expense file is moved to quarantine; a missing one is recorded, and the P&L still runs with those vehicles marked as missing costs.

One Airflow DAG, `fleet_daily_reconciliation`, checks every real minute and batches each simulated day once, about a minute after the day ends; the other runs stop at the first task because the day is already done. It takes the date from the simulated clock, not from Airflow's own date (the two are unrelated, and using Airflow's would process the wrong folder on every run). It saves a snapshot of the speed view, checks the expense file, branches on the result, runs both Spark jobs, checks that the high-water mark moved, and records the difference between the speed and batch views. The DAG file contains no business logic; a test that parses it fails if any appears. The runs are shown in Figure 16 and the task graph in Figure 23.

6.5. Serving layer

A FastAPI service is the only way to read the data. It reads Redis and PostgreSQL through connection pools, applies the merge rule, and says in every answer which rows are exact and which are approximate. Grafana reads through the same API, not from the stores directly, so a broken panel is a true report about the serving layer.

```
curl: the merged answer, with where each day came from
$ curl -s '/api/v1/fleet/utilization?from=2026-03-03&to=2026-03-06' (rows counted per date and source)

batch_complete_thru: 2026-03-05
consistency: batch-complete-through 2026-03-05
uncovered_dates: []
degraded: False
row_count: 47
2026-03-03 source=batch exact=True zones=12
2026-03-04 source=batch exact=True zones=12
2026-03-05 source=batch exact=True zones=12
2026-03-06 source=speed exact=False zones=11
```

Figure 8: The merged answer for a range of dates. Dates up to the high-water mark come from the batch view, later dates from the speed view.

7. Observability

What	How	Why
Ingestion rate and errors	Counters in the producers, scraped by Prometheus	If this is zero, every other panel looks calm only because there is no traffic
Injected faults	Counter per fault type	The known input for checking the dead-letter path
Simulated clock drift	Gauge per container	Two services disagreeing about the date break every window
Streaming progress	A Spark listener pushes to Pushgateway	The only way to tell a stopped query from an idle one (below)
Micro-batch duration	Same listener	A duration near the trigger means batches start late and delay grows
Sink writes and errors	Counter in each writer	A failing writer is how the live view goes stale while all looks fine
High-water mark age	Gauge set by the API	A stopped batch layer has no other symptom
Degraded answers	Counter per missing store	One store down is allowed, but should not last
API latency	FastAPI instrumentator	A rising p95 with a flat p50 points to one slow store
Logs	<code>structlog</code> JSON in every service	One format everywhere; reserved field names are refused

Table 5: What is measured, how, and why.

7.1. Consumer lag does not work for Structured Streaming

The usual way to watch a Kafka consumer is consumer-group lag. **It does not work here.** Structured Streaming keeps its offsets in its own checkpoint and never commits them to a consumer group, so the lag reads zero whether a query is keeping up or has been dead for hours. On the running system, listing consumer groups returns none at all. An earlier attempt to force it with `kafka.group.id` stopped the dead-letter query after about 4,500 micro-batches.

What works is the **age of the last progress report** pushed by a Spark listener. A query that processed zero rows may be healthy and idle; a query that has stopped reports nothing, and only the age of the last report tells the two apart. Pushgateway keeps the last value after the job dies, so we alert on the age of the timestamp, not on its value.

7.2. An alert that could not fire

The brief asks for an alert when no data has arrived for some minutes. Our first version was:

```
sum(rate(events_produced_total[2m])) == 0
```

It loaded without error. We then stopped the producer, and the alert stayed inactive for six and a half minutes. The reason: when the producer dies, its metrics disappear, and a sum over nothing is not zero but an empty result, which never equals zero. The alert for “no data” was defeated by having no data. The fix adds `or vector(0)`, plus a second rule on `up == 0`, a series Prometheus always keeps for every target. The measured times after the fix are in Section 8.

7.3. Inhibition

Stopping the producer makes several rules true at once: the producer target is down, no telemetry arrives, and the streaming queries stall. Only the first is the cause. Alertmanager inhibition hides the consequences so the person on call sees the cause. Our first inhibition rule hid nothing, because it matched `severity="warning"` while the alerts it should hide are `critical`. It now follows the chain from cause to effect. Every scrape target is shown up in Figure 17.

8. Results

All numbers in this section come from one clean run started on 27 September 2026 and are recorded, with how each was measured, in `docs/evidence/rerun-measurements.md`.

8.1. The business question, answered

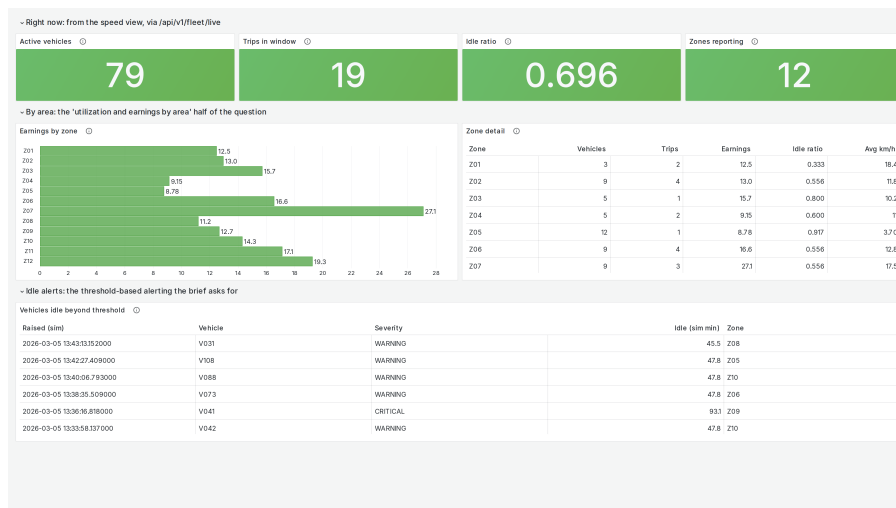


Figure 9: Question A: live utilization, idle ratio and earnings by zone, served through the API.

The same list, read straight from PostgreSQL, is in Figure 18 in the appendix.

8.2. Checking against answers we already knew

The strongest evidence is that two independent calculations agree. Revenue per vehicle is summed from the trip join, grouped by vehicle. Earnings per zone are summed with a different rule for picking one row per trip, grouped by zone and hour. The two totals must be equal for every day.

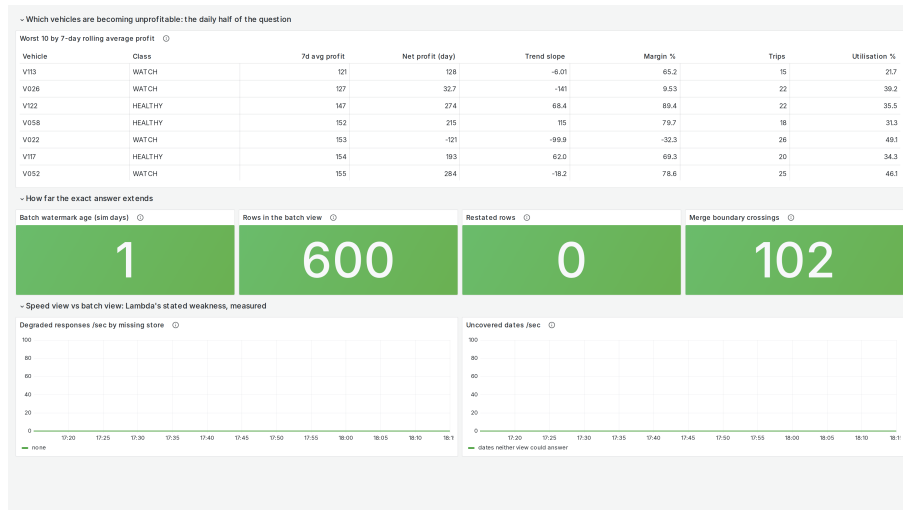


Figure 10: Question B: vehicles ranked by 7-day rolling average profit, with their label and trend. Ranking by the rolling average, not by today, stops one bad day from outranking a vehicle that has lost money all week.

Simulated date	P&L revenue	Zone earnings	Difference
2026-03-01	47,664.77	47,664.77	0.00
2026-03-02	49,644.29	49,644.29	0.00
2026-03-03	49,969.45	49,969.45	0.00
2026-03-04	50,707.43	50,707.43	0.00
2026-03-05	50,456.36	50,456.36	0.00

Table 6: Two independent paths to the same revenue.

The second check is the dead-letter path. The producer counts the faults it injects; the stream rejects readings on physical rules only. With the producer stopped, so both counts were fixed, they compared as follows:

Fault type	Injected	Dead-lettered
Missing coordinates	1,415	1,415
Position outside the city	922	922
Negative fare	974	975
Impossible speed	907	907
Timestamp two hours ahead	474	475
Total	4,692	4,694

Table 7: Faults injected by the simulator against readings rejected by the stream.

Three of the five types match exactly. Negative fare differs by 1. Timestamp two hours ahead differs by 1. The injected counts were read about a second before the producer stopped, so a fault sent in that second is dead-lettered but not in the snapshot.

Finally, the vehicle registry topic holds 150 vehicles, one per vehicle in the fleet, and after 5 simulated days the batch layer had written 750 P&L rows, which is 5 times 150.

8.3. The merge, and what happens when a store fails

A request for 2026-03-02 to 2026-03-05, with the high-water mark at 2026-03-04, returned 48 rows: 36 from the batch view and 12 from the speed view. Every date came from exactly one side. We then stopped the stores one at a time.

While Redis was stopped the streaming job kept running and kept writing the lake (no restarts), and the live view came back on its own when Redis returned. The degraded answer itself is shown

Situation	HTTP	Answer
Both stores up	200	36 batch, 12 speed rows, not degraded
Redis stopped	200	36 batch rows, X-Data-Degraded: true , Redis named as missing
Both stopped	503	names both stores; never a 500
PostgreSQL stopped	200	12 speed rows, X-Data-Degraded: true , PostgreSQL named as missing; high-water mark reported as unknown
Both restarted	200	36 batch, 12 speed rows, not degraded

Table 8: One store down is a warning, not an outage. Only losing both fails, and it fails with a clear 503, never a 500.

in Figure 19 in the appendix.

8.4. Performance

Measurement	Value	Target
Ingestion rate	239.9 events/s	240 configured
Live view freshness, p50 and p95 (real)	9.1 s and 23.2 s	NFR-1: under 30 s, met
API latency, p50, p95 and p99	6 / 18 / 29 ms	
Both batch jobs for one simulated day	53 to 56 s	
Airflow run that batches a day, mean and longest	91 s and 105 s	5 min per simulated day
First alert firing (producer target down)	76 s	NFR-4: under 3 min, met
“No data received” alert firing	181 s	
All alerts resolved after the restart	19 s	
Memory, whole stack	4.8 GiB	
Automated tests	461 passed	

Table 9: Measured on the running stack.

Streaming query	p50 (s)	p95 (s)
q_master	1.5	3.4
q_dlq	0.6	3.2
q_vehicles	2.0	4.8
q_activity	3.7	6.2
q_earnings	3.7	7.9
q_idle	5.9	12.8

Table 10: Micro-batch duration per query over 20 real minutes, against a 10 s trigger.

At the 95th percentile `q_idle` takes longer than the 10 second trigger, so its next batch starts late. It is the stateful idle detector, the heaviest query. The other five stay inside the trigger. On average the speed layer keeps up; at the tail the stateful query does not, which is the main reason freshness at p95 is well above p50.

8.5. Corrections and running twice

We ran the batch layer again for one past date while the high-water mark stood at a later date. `restatement_count` is summed over that date’s 150 rows.

8.6. Failure test: the producer stops

We stopped the telemetry producer with `make chaos-kill-producer` and brought it back with `make chaos-heal`. The alert fired and resolved as shown in the performance table, and Alertmanager showed only `ProducerTargetDown`, with `NoTelemetryIngested` inhibited as a consequence (Figure 20).

Run	Job run id	Sum of restatement_count
before		0
A	new	150 (all 150 rows recorded as corrected)
B	same as A	150 (no change)

Table 11: A second run with the same job id changes nothing. A run with a new id is recorded as a correction. The high-water mark never moved backwards.

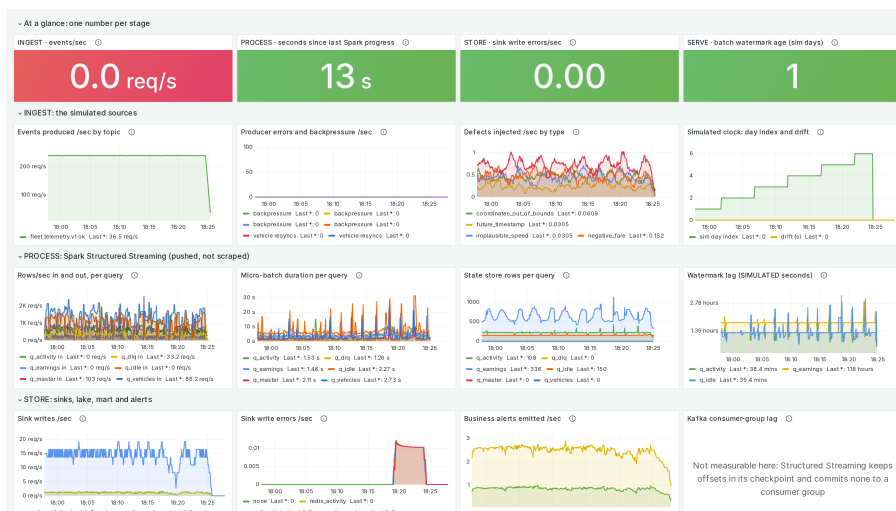


Figure 11: Pipeline health during the producer outage: ingestion drops to zero while the streaming queries stay alive and report idle. The short bump in sink write errors is the earlier Redis stop, now counted instead of stopping the job.

9. Limitations and lessons

9.1. Every bug we found was silent

Not one bug in this project announced itself. None raised an exception, none appeared in a log, and each one would have survived a demonstration that only asked “is it running?”. Earlier examples: revenue was 8.5 times too high because the fare repeats on every reading of a trip; the speed layer started only two of its six queries, so Redis was simply empty; an Airflow branch skipped the whole batch path and the DAG went amber, not red; and the Airflow web UI was down for three hours while the scheduler kept running.

The final clean runs for this report found eight more, most of them by comparing an output with a number we already knew:

- **Vehicles drove out of the city.** More “outside the city” readings were dead-lettered than injected: pickup points and position noise were not kept inside the area the validator accepts. They now are, with a test over a whole simulated day.
- **The future-timestamp check moved with the load.** It used the micro-batch clock, which is early or late by however far processing is behind: 98 honest readings were rejected in one run, and 15 of 480 injected faults passed in another. Readings are now compared with the time they were sent; honest ones are within one simulated minute of it, the injected fault two hours ahead.
- **A skipped snapshot skipped the whole day.** With no zones in the speed view yet, the snapshot step skipped, and the default trigger rule passed the skip down the batch path. The branch now runs unless something actually failed.
- **Yesterday was answered by neither view.** On a fixed 5 minute timer, every DAG run started just before the simulated midnight, so the high-water mark stayed two days behind for all but about 15 seconds of each day. The DAG now checks every minute; over 20 minutes the mark was at yesterday in 81 of 113 (72%) of samples.
- **The trip de-duplication never forgot a trip.** Spark only clears that state when the time column is in the key. It now uses `dropDuplicatesWithinWatermark`, with a streaming test that

old trips are cleared.

- **A Redis outage stopped the whole speed layer.** A Redis error inside a writer ended its query, the job stopped, and the container restarted in a loop, which also paused the lake writes. The writers now skip the batch and count the error.
- **A false alarm after every outage.** Spark's idle message has the query's id but not its name, so heartbeats were sent as "unnamed" and raised a stall alert once data resumed. The listener now keeps each query's name.
- **Two dashboards showed "No data".** An unpinned Grafana plugin updated to a version our Grafana cannot load, and the screenshot script still reported success. The plugin is pinned and the script now rejects "No data".

After these fixes, the dead-lettered counts in Table 7 match the injected ones, apart from faults sent in the second before the producer stopped. The lesson we take from this project: **a pipeline that merely runs tells you nothing; a pipeline whose output can be checked against a number you already know tells you a lot.** The same applies to the checks themselves. Our screenshot script once reported success for a login page, the alert rule above loaded and could never fire, and a DAG test passed against zero DAGs. Each was fixed by making the check test something it could get wrong.

9.2. Simplifications

Scale. One Kafka broker with replication factor 1; telemetry kept in Kafka for one real hour; Spark in local mode with four cores (windows, watermarks, checkpoints and state behave as on a cluster, but work is not spread over machines); 150 vehicles; no authentication or TLS, and default passwords. At 240 events per second this is not a test of scale; it shows that the architecture is correct and how the pipeline behaves.

Correctness. The system is at-least-once, so every writer overwrites instead of adding. Speed-layer distinct counts are approximate (HyperLogLog, about 2 % error). Live earnings lag by up to one trip, because a fare counts when the trip ends. Readings later than the 30 simulated minute watermark are left out of the live windows but still reach the lake. The lake has no transactions, so a crashed writer can leave a partial file.

9.3. The speed and batch views differ more than we predicted

Every day the DAG saves a snapshot of the speed view and later compares it with the batch figures for the same hour. We predicted a difference of 1 to 3 % on counts and up to 8 % on earnings. Over 3 days, the idle ratio agreed within 10.9 %, but trips per minute read 70 to 79 % low and earnings per minute 24 to 43 % low.

Two real bugs were fixed on the way to these numbers. The first comparison put a 15 minute sliding window next to a whole batch day. The second matched rows by when Airflow ran, not by the time window the data described. What remains is not a fault in either layer: the snapshot catches a sliding window that is still filling, so counts and earnings read low, while a ratio such as the idle ratio does not depend on how full the window is. That is why the idle ratio agrees and the counts do not. The real check that the two layers compute the same thing is the revenue comparison in Section 8. Making counts comparable would need the live view to keep a full history per hour, which we leave as future work.

9.4. What is not implemented

- **Distributed tracing.** The design planned OpenTelemetry and Jaeger; it was cut for time. Observability rests on logs, metrics and alerts.
- **A PDF daily report.** The brief accepts "a consolidated report or dashboard"; the Grafana dashboards meet it.
- **Automatic corrections.** The expense dropper writes a marker when it sends a corrected file, but nothing reads it. A correction is applied by running `make dag-trigger DATE=...` for that

date.

- **Catching up missed days.** The DAG does not run days missed during an outage (**catchup** is off); **make backfill** does that instead.

The most important simplification: the expense file is generated from *our own* simulated distance, so the two sources are not truly independent. A real cost source could disagree in ways this simulation cannot produce, and the reconciliation results should be read with that in mind.

9.5. At production scale

Kafka with three replicas, TLS, authentication and schema checks in CI; Spark on Kubernetes with a RocksDB state store and *separate clusters for speed and batch*, so a batch job cannot slow the live view; Delta Lake or Iceberg on the lake, so a correction becomes a **MERGE** with full history; data quality tests with quarantine; and access control, lineage and GDPR handling for driver data, which a system that computes profit per driver clearly needs.

10. Conclusion

The system takes in two different sources, processes them on a speed path and a batch path that share one tested transform library, joins the two at one clear boundary, and shows its own health. Both halves of the business question are answered from live data: utilization and earnings by area within seconds, and a ranked list of vehicles becoming unprofitable once partner costs are counted.

We chose Lambda because the question has two halves with opposite needs, and because a corrected invoice should cost work in proportion to the correction, not to a whole day of telemetry. Six of the eight criteria support that choice; the two that do not are accepted and reduced in the code.

The main lesson: every bug we found was silent, and the one method that found them was checking an output against a number we already knew. Two independent calculations of the same revenue agreeing to the cent say more about whether this pipeline is correct than any number of green dashboard panels.

References

- [1] EC8203 Applied Big Data Engineering, lecture notes on Lambda and Kappa architectures, streams and queues, and data governance.
- [2] Apache Kafka documentation: design, partitions and ordering, KRaft mode.
- [3] Apache Spark, *Structured Streaming Programming Guide*: event-time windows, watermarks and `foreachBatch`.
- [4] Apache Airflow documentation: trigger rules, branching and sensors.
- [5] Prometheus documentation: alerting rules and the limits of Pushgateway.
- [6] Confluent documentation: Schema Registry and the Avro wire format.

A. Individual contributions

All three members took part in testing, the demonstration and reviewing this report.

B. Reproducing the results

C. More screenshots

Member	Major parts
Jathurshan	Architecture decision and the Lambda versus Kappa argument; simulated clock and shared configuration; stateful idle detection; the serving layer and the merge boundary; the architecture diagrams.
Shamil	Storage design (lake layout, star schema, Redis keys); the batch layer (zone hourly figures and daily profitability per vehicle); corrections and idempotency; the Airflow DAG.
Rifat	Simulated sources and ingestion; Avro and the Schema Registry; the speed layer queries and writers; observability (metrics, alert rules, dashboards) and the screenshots.

Command	What it does
<code>make clean</code>	Stop everything and delete all data, for a fresh simulated day 1
<code>make up-obs</code>	Start the pipeline, Airflow, and the monitoring stack
<code>make backfill DAYS=4</code>	Run the batch layer over the last complete simulated days
<code>make batch-check</code>	Print the two-path revenue check
<code>make reconcile DATE=...</code>	Speed versus batch difference for one date
<code>make chaos-kill-producer</code>	Stop ingestion; the alert fires
<code>make chaos-heal</code>	Bring it back; the alert resolves
<code>make test</code>	The automated test suite
<code>make diagrams, make figures</code>	Rebuild the diagrams and screenshots

The screenshot shows the Apache Kafka UI for the topic 'fleet.telemetry.v1'. The 'Messages' tab is selected, showing a list of messages. The first message is highlighted, showing its details. The message is a JSON object with the following fields: event_id, vehicle_id, driver_id, lat, lon, speed_kph, status, event_time, ingest_time, producer_id, and schema_version. The key is 'V016'.

Figure 12: Kafka UI: telemetry decoded from Avro through the Schema Registry. The key is the vehicle.

The screenshot shows the Spark UI for the 'Structured Streaming' application. The 'Streaming Query' section is active, displaying a table of active streaming queries. The table has columns for Name, Status, ID, Run ID, Start Time, Duration, Avg Input Rate, Avg Process Rate, and Latest Batch. The queries are related to the speed layer, including vehicles, idle, earnings, activity, dtq, and master.

Figure 13: Spark UI: the six streaming queries of the speed layer.

```
redis-cli: one zone in the speed view
$ redis-cli --scan --pattern 'fleet:zone:*'; redis-cli HGETALL <first zone>

fleet:zone:Z03
fleet:zone:Z08
fleet:zone:Z12
fleet:zone:Z06
fleet:zone:Z11

HGETALL fleet:zone:Z03
active_vehicles 8
trips 2
idle_ratio 0.6250
avg_speed_kmh 14.3
window_start 2026-03-06 04:05:00
window_end 2026-03-06 04:20:00
updated_at_sim 2026-03-06 04:20:00
earnings 9.69
completed_trips 1
earnings_window_end 2026-03-06 05:00:00
```

Figure 14: The speed view in Redis: one hash per zone, overwritten every micro-batch and expiring after two simulated hours.

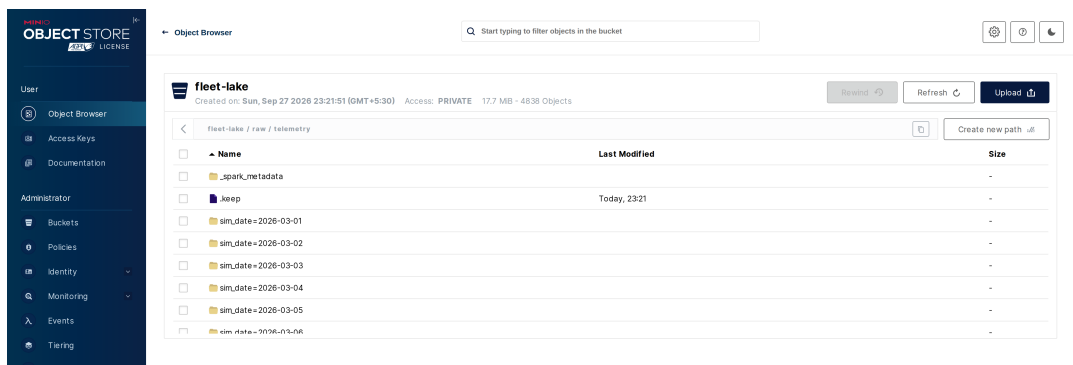


Figure 15: MinIO: the raw telemetry in the lake, one folder per simulated date.

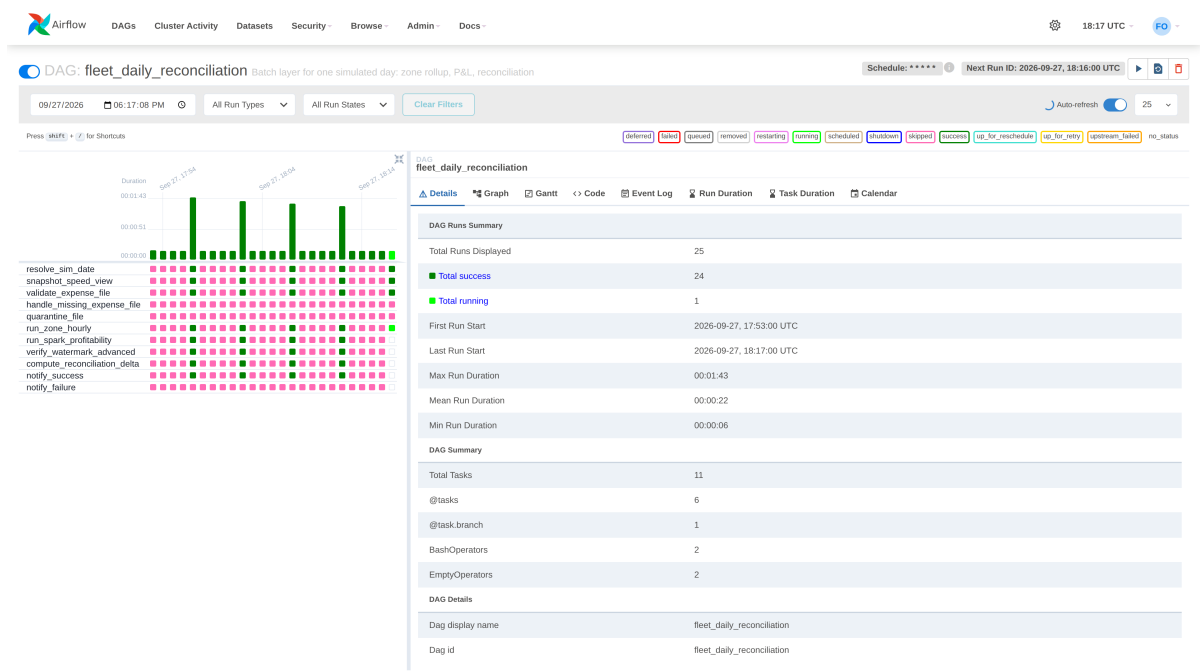


Figure 16: Airflow: one run a minute. The long runs batch a finished day; the short ones find the day already done and stop.

Prometheus Alerts Graph Status Help

Targets

All scrape pools **All** Unhealthy Collapse All Unknown Unhealthy Healthy

api (1/1 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://api:8000/metrics	UP	component="telemetry-api" instance="api:8000" job="api" scrape="server"	674.000ms ago	11.797ms	

kafka (1/1 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://kafka-exporter:9308/metrics	UP	component="kafka" instance="kafka-exporter:9308" job="kafka" scrape="agent"	456.000ms ago	41.939ms	

producers (2/2 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://telemetry-producer:8001/metrics	UP	component="telemetry-producer" instance="telemetry-producer:8001" job="producers" scrape="agent"	2.60s ago	8.804ms	
http://expense-dropper:8002/metrics	UP	component="expense-dropper" instance="expense-dropper:8002" job="producers" scrape="agent"	2.374s ago	6.880ms	

prometheus (1/1 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://localhost:9090/metrics	UP	instance="localhost:9090" job="prometheus" scrape="server"	4.787s ago	9.682ms	

pushgateway (1/1 up) [View logs](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://pushgateway:9091/metrics	UP	instance="pushgateway:9091" job="pushgateway" scrape="process"	893.000ms ago	5.480ms	

Figure 17: Prometheus: every scrape target up.

```
psql: the five least profitable vehicles on the latest batch day
$ psql fleet_mart: the five lowest net_profit rows on the latest sim_date
```

sim_date	vehicle_id	trips	revenue	costs	net_profit	classification
2026-03-04	V022	26	375.13	496.36	-121.23	WATCH
2026-03-04	V039	26	368.49	403.60	-35.11	WATCH
2026-03-04	V026	22	342.61	309.95	32.66	WATCH
2026-03-04	V087	24	349.07	293.65	55.42	WATCH
2026-03-04	V113	15	196.31	68.29	128.02	WATCH

(5 rows)

Figure 18: The same answer straight from the PostgreSQL mart: the least profitable vehicles on the latest batch day.

```
curl: the same request with Redis stopped
$ curl -si '/api/v1/fleet/utilization?from=2026-03-03&to=2026-03-06' (status, header, envelope)

HTTP/1.1 200 OK
x-data-degraded: true
batch_complete_thru: 2026-03-05
consistency: batch-complete-through 2026-03-05; DEGRADED - unreachable: redis
degraded: True
missing_stores: ['redis']
uncovered_dates: []
row_count: 36
```

Figure 19: With Redis stopped the API still answers from the batch view, and says so in a header and in the body.

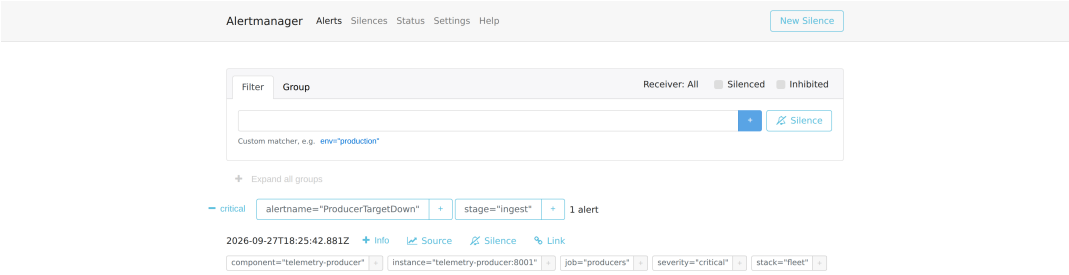


Figure 20: Alertmanager during the outage: the cause is shown and its consequences are inhibited.

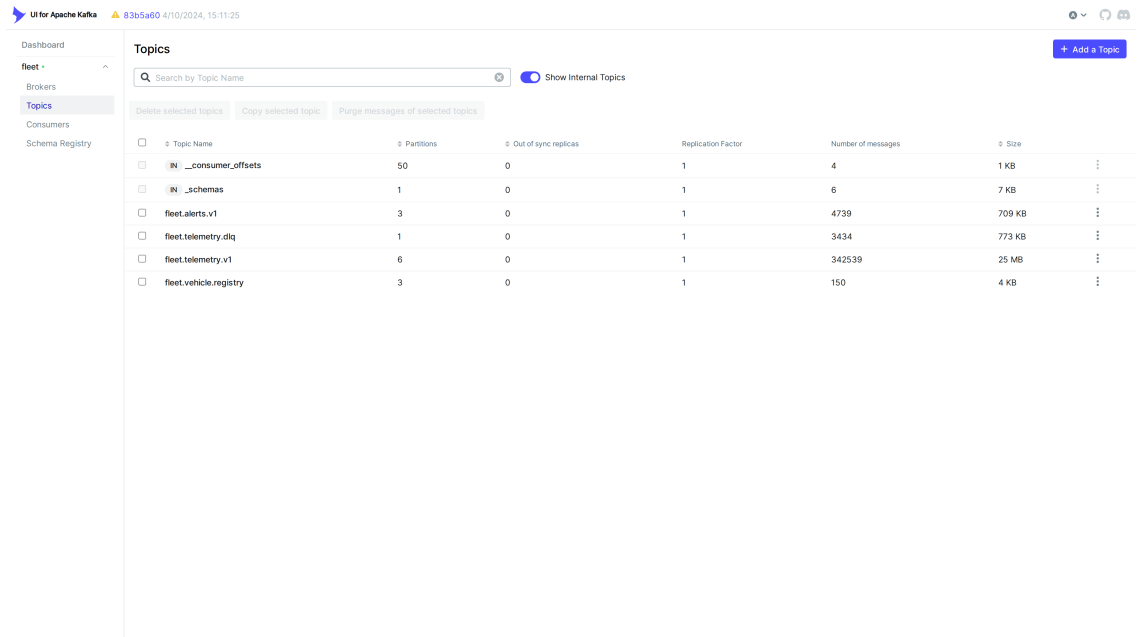


Figure 21: Kafka UI: the topics and their message counts.

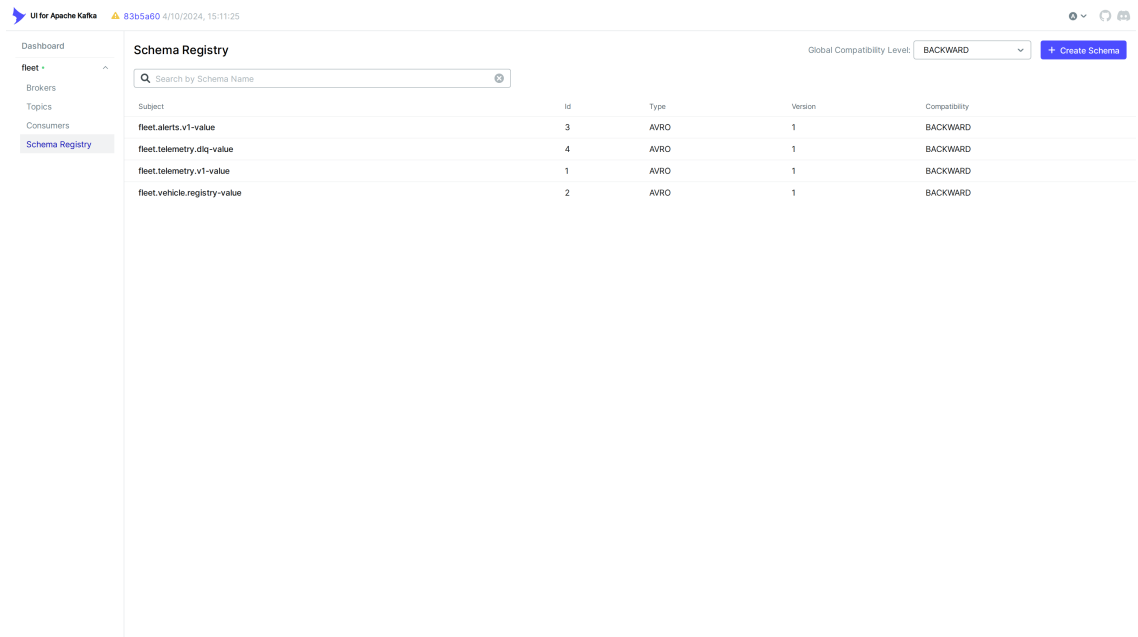


Figure 22: Schema Registry: the four Avro subjects.

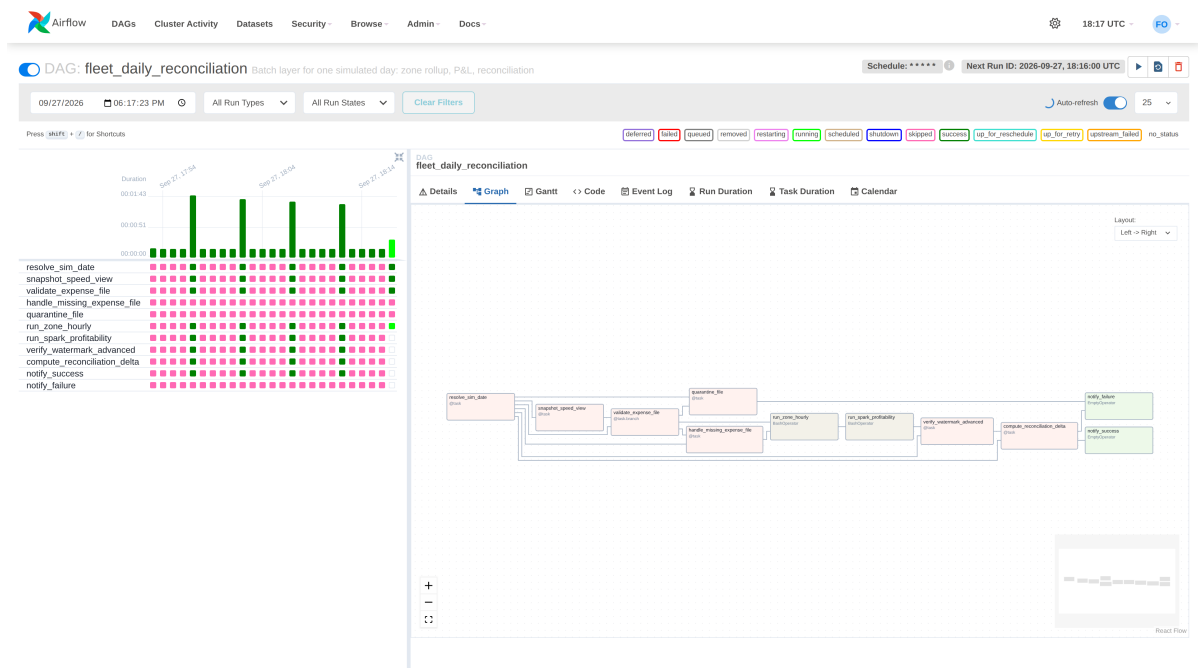


Figure 23: Airflow: the DAG graph, with the branch on the expense file.

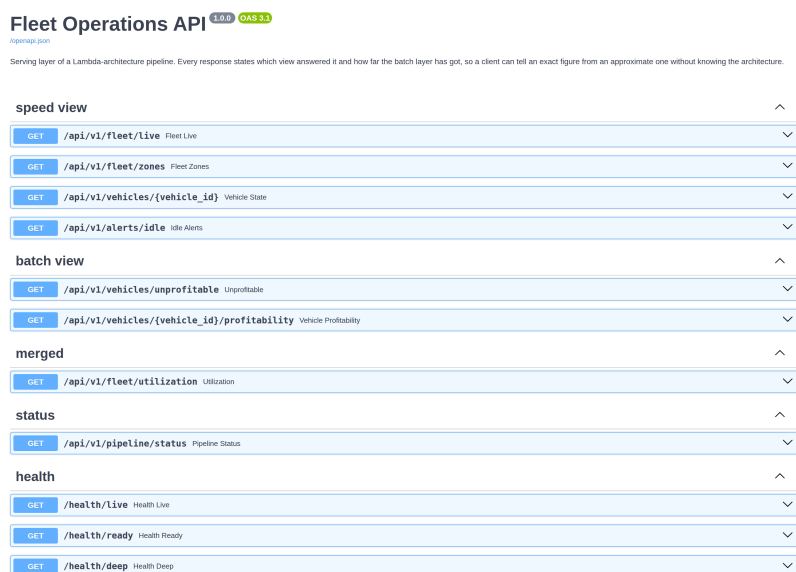


Figure 24: The API contract (OpenAPI).

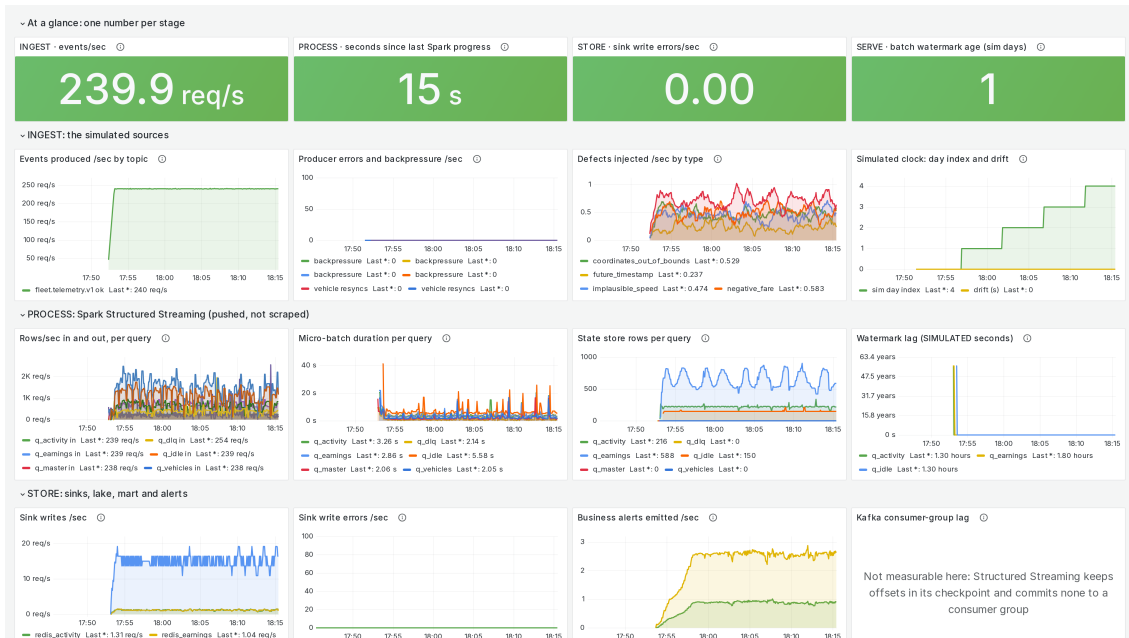


Figure 25: Pipeline health in normal running.

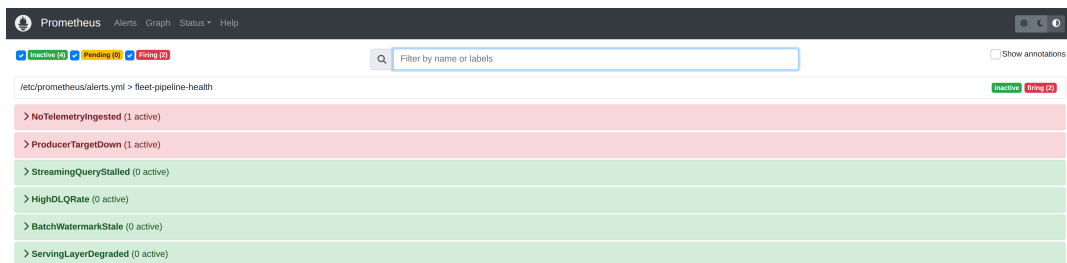


Figure 26: Prometheus during the producer outage: two of the six alert rules firing.

```

make batch-check: two independent paths to the same revenue
$ make -s batch-check

pnl rows          : 750
distinct sim_dates: 5
restated rows     : 0
zone-hourly rows  : 1428
watermark         : 2026-03-05

classification by sim_date:
  sim_date | classification | count
-----+-----+-----
2026-03-01 | INSUFFICIENT_DATA | 150
2026-03-02 | INSUFFICIENT_DATA | 150
2026-03-03 | INSUFFICIENT_DATA | 150
2026-03-04 | HEALTHY           | 92
2026-03-04 | WATCH            | 58
2026-03-05 | HEALTHY           | 90
2026-03-05 | WATCH            | 60
(7 rows)

revenue vs zone earnings (two independent paths to one number - must match):
  sim_date | pnl_revenue | zone_earnings | delta
-----+-----+-----+-----
2026-03-01 | 47664.77 | 47664.77 | 0.00
2026-03-02 | 49644.29 | 49644.29 | 0.00
2026-03-03 | 49969.45 | 49969.45 | 0.00
2026-03-04 | 50707.43 | 50707.43 | 0.00
2026-03-05 | 50456.36 | 50456.36 | 0.00
(5 rows)

```

Figure 27: make batch-check: the rows written and the two revenue paths side by side.